Dependency Injection

DIP, IoC, Guice, testability — Advanced Programming

Ayoub FRIHAOUI

ayoub.frihaoui@lecnam.net
Conservatoire national des arts et métiers (CNAM Paris)

Original material © Rezak AZIZ, CNAM Paris.
Source: `https://rezakaziz.github.io/PROG-AVANC/`

Context: software architecture

General context

A software application is made of several components (classes, modules or services) that interact to provide a global feature.

Context: software architecture

General context

A software application is made of several components (classes, modules or services) that interact to provide a global feature.

Observation

These components often depend on each other to work.

User interface

↓

Business services

↓

Data access

The dependency problem

Fundamental problem

Bad dependency management leads to excessive coupling between the components of the system.

The dependency problem

Fundamental problem

Bad dependency management leads to excessive coupling between the components of the system.

Consequences

- hard maintenance;
- weak evolvability;
- complex unit tests;
- limited reusability.

Notion of coupling

Definition

A class **A** is said to be **coupled** to a class **B** when the code of **A** explicitly depends on the implementation of **B**.

Notion of coupling

Definition

A class **A** is said to be **coupled** to a class **B** when the code of **A** explicitly depends on the implementation of **B**.

Definition

A class **A** is said to be **strongly coupled** to a class **B** when **A** is coupled with **B** and **B** is coupled with **A**.

Notion of coupling

Definition

A class **A** is said to be **coupled** to a class **B** when the code of **A** explicitly depends on the implementation of **B**.

Definition

A class **A** is said to be **strongly coupled** to a class **B** when **A** is coupled with **B** and **B** is coupled with **A**.

Chained dependencies

Class A $\rightarrow$ Class B $\rightarrow$ Class C

A indirectly depends on C

Notion of coupling

Definition

A class **A** is said to be **coupled** to a class **B** when the code of **A** explicitly depends on the implementation of **B**.

Definition

A class **A** is said to be **strongly coupled** to a class **B** when **A** is coupled with **B** and **B** is coupled with **A**.

Chained dependencies

Class A $\rightarrow$ Class B $\rightarrow$ Class C

A indirectly depends on C

Coupling

A is coupled to B $\Leftrightarrow$ compiling A requires compiling B.
Any change to B directly impacts A.

Central question

Problem

How to design loosely coupled software while keeping a clear, maintainable architecture?

Central question

Problem

How to design loosely coupled software while keeping a clear, maintainable architecture?

Why reduce coupling?

- Divide and conquer
- Simplify creating unit tests.
- Simplify teamwork.

Coupling example

Source code

```
public class Calculator {
    private FileLogger logger;

    public Calculator() {
        logger = new FileLogger("log.txt");
    }

    public int compute(int x) {
        logger.log("compute " + x);
        return 2 * x;
    }
}
```

Coupling example

Source code

```
public class Calculator {
    private FileLogger logger;

    public Calculator() {
        logger = new FileLogger("log.txt");
    }

    public int compute(int x) {
        logger.log("compute " + x);
        return 2 * x;
    }
}
```

Analysis

- concrete dependency;
- internal instantiation;
- frozen choice;
- reduced testability.

Section 1

Dependency Inversion Principle

Dependency inversion principle

Goal

Reduce coupling between the components of a software system by removing direct dependencies on concrete implementations.

Dependency inversion principle

Goal

Reduce coupling between the components of a software system by removing direct dependencies on concrete implementations.

Central idea

A component must not depend on another concrete component, but on an abstraction.

Business component

↓

Abstraction

↑

Implementation

Dependency Inversion Principle (SOLID)

Statement of the principle

- High-level modules must not depend on low-level modules.
- Abstractions must not depend on details. Details must depend on abstractions.

Consequence

Abstractions define the collaboration contracts between components.

Dependency Inversion Principle (SOLID)

Statement of the principle

- High-level modules must not depend on low-level modules.
- Abstractions must not depend on details. Details must depend on abstractions.

Consequence

Abstractions define the collaboration contracts between components.

Business component

↓

Abstraction

↑

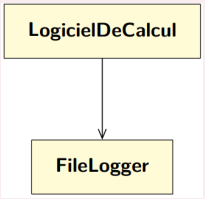
Implementation

Programming against interfaces

Principle

An interface defines *what* a component does, without saying *how* it does it.

Before DIP

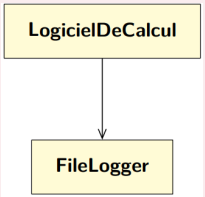


Programming against interfaces

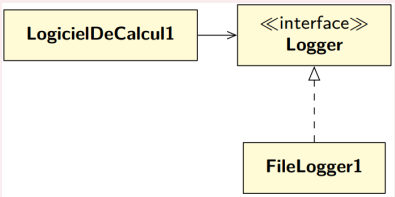
Principle

An interface defines *what* a component does, without saying *how* it does it.

Before DIP



After DIP



Advantages

code decoupling; implementation interchangeability; better evolvability.

Example: abstraction through an interface

Interface

```
public interface Logger {
    void log(String message);
}
```

Implementation

```
public class FileLogger
    implements Logger {
    ...
}
```

Analysis

- the contract is explicit;
- several implementations possible;
- the client code is independent.

Limit of the interface alone

Problem

Using an interface is not enough if the client code directly instantiates a concrete implementation.

Limit of the interface alone

Problem

Using an interface is not enough if the client code directly instantiates a concrete implementation.

Consequence

Coupling is merely moved, not removed.

Problematic case

```
public class Calculator {
    private Logger logger;

    public Calculator() {
        logger = new FileLogger("log.txt");
    }

    public int compute(int x) {
        logger.log("compute " + x);
        return 2 * x;
    }
}
```

Section 2

Inversion of Control Principle

Putting into perspective

Reminder: dependency inversion

- Less coupling between components;
- Dependence on abstractions;
- Separation between contract and implementation.

Identified limit

- DIP does not say:
  - who controls execution;
  - when components are called.
- The control problem remains whole.

Central problem

Fundamental question

Who should be responsible for controlling the overall flow of the application?

Architectural stakes

- System maintainability;
- Evolvability;
- Clear separation of responsibilities.

Classical organisation

How it works

In a traditional architecture, the business component:

- creates its dependencies;
- directly calls the other components;
- controls the execution order.

Consequences

- Strong coupling;
- Business logic mixed with orchestration;
- Architecture hard to evolve.

Inversion of control principle

Definition

Inversion of control is an architectural principle where control of the execution flow is no longer handled by the business component, but delegated to an external infrastructure.

Inversion of control principle

Definition

Inversion of control is an architectural principle where control of the execution flow is no longer handled by the business component, but delegated to an external infrastructure.

Central idea

- The business component becomes passive;
- It no longer decides when it runs;
- It focuses on its functional logic.

Architectural effects

Before IoC

- Local orchestration;
- Frozen dependencies;
- Implicit execution flow.

After IoC

- Centralised orchestration;
- Independent components;
- Explicit execution flow.

Conceptual reading

Without inversion of control

`Business component` → `controls the flow`

Characteristic

The component knows the overall flow of the application.

With inversion of control

`Infrastructure` → `orchestration`

Characteristic

The component ignores the global context in which it runs.

Hollywood principle

Statement

Don't call us, we'll call you

Interpretation

- The component does not call the infrastructure;
- It exposes extension points;
- It is invoked at the right time.

Inversion of control and frameworks

Observation

Frameworks:

- control the life cycle;
- orchestrate execution;
- call the application code.

Conclusion

A framework is not a library. It imposes an execution framework based on IoC.

Section 3

Dependency injection

Transition

Reminder

Inversion of control entrusts control of the execution flow to an external infrastructure.

Question

How to concretely provide dependencies to components without them creating the dependencies themselves?

Dependency injection: definition

Definition

Dependency injection is a design mechanism in which an object's dependencies are provided from the outside, rather than created by the object itself.

Goal

- Remove the responsibility of creating dependencies;
- Complete the inversion of control.

Conceptual comparison

Without injection

`Object` → `new Dependency()`

Characteristic

The object controls creating its dependencies.

With injection

`Object` ← `Dependency`

Characteristic

Dependencies are provided by an external entity.

Role of dependency injection

Removed responsibilities

- Creating objects;
- Choosing implementations;
- Technical configuration.

Kept responsibility

- Business logic;
- Functional rules;
- Using dependencies.

Injection mechanisms

Main mechanisms

- Constructor injection;
- Setter injection;
- Mixed approach.

Choice criterion

The mechanism depends on whether the dependency is mandatory or optional.

Constructor injection

Principle

Dependencies are provided when the object is created.

Example

```
public class Calculator {
    private Logger logger;

    public Calculator(Logger logger) {
        this.logger = logger;
    }

    public int compute(int x) {
        logger.log("compute " + x);
        return 2 * x;
    }
}
```

Characteristics

- Explicit dependencies;
- Object valid from creation;
- Recommended approach.

Setter injection

Principle

Dependencies are provided after the object is created, through dedicated methods.

Example

```
public class Calculator {
    private Logger logger;

    public void setLogger(Logger logger) {
        this.logger = logger;
    }

    public int compute(int x) {
        logger.log("compute " + x);
        return 2 * x;
    }
}
```

Warning

- Partially initialised object;
- Potentially missing dependencies.
- Reserve for genuinely **optional** dependencies with a safe default; mandatory ones go through the constructor.

Mixed approach

Principle

A dependency is provided at construction, but can still be replaced later (e.g. in tests).

Point

Compromise between safety and flexibility.

Example

```
public class Calculator {
    private Logger logger;

    public Calculator(Logger logger) {
        this.logger = logger;
    }

    public void setLogger(Logger logger) {
        this.logger = logger;
    }

    public int compute(int x) {
        logger.log("compute " + x);
        return 2 * x;
    }
}
```

Comparing mechanisms

Summary

- Constructor:
  - safe;
  - explicit;
  - recommended.
- Setter:
  - flexible;
  - less safe.

Practical rule

A mandatory dependency must be injected through the constructor.

Injection frameworks

Observation

In a real application, manual object creation quickly becomes complex:

- many dependencies;
- cascading dependencies;
- varying configurations.

Limit

Assembly code becomes hard to maintain and evolve.

Manual assembly

```
new A(new B(new C()))
new A(new B(new D()))
...
```

Role of an injection framework

Main responsibilities

A dependency injection framework:

- creates objects;
- resolves dependencies;
- injects the right components;
- manages the object life cycle.

Conceptual view

Application

↓

Injector

↓

Configured objects

Notion of assembler

Definition

The assembler is the component in charge of building and wiring the application's objects.

Evolution

In a framework, the assembler is automated and configurable.

Responsibility

Creation
Injection
Configuration

Configuring dependencies

Principle

The framework must know which implementation to associate with each abstraction.

Common approaches

- external configuration (files);
- annotations in the code;
- conventions.

Framework examples

Known frameworks

- Spring (Java ecosystem);
- Guice (lightweight, DI-focused).

Positioning

DI = foundation

Framework = automation

Remark

These frameworks go far beyond dependency injection.

What the framework really does

Important

An injection framework:

- does *not* improve a bad design;
- is only useful if the principles are respected.

Warning

Using a framework without understanding dependency injection leads to complex, fragile code.

Synthesis

To remember

- DI is a mechanism;
- It implements IoC;
- It improves testability;
- It prepares framework use.

Key message

Good design is code that does not know how its dependencies are created.

Section 4

Dependency injection and testability

Testability: the problem

Problem

Strongly coupled code is hard to test in isolation, as it depends on external resources (files, databases, network).

Consequence

Tests become slow, fragile and hard to reproduce.

What injection brings

Dependency injection lets you:

- replace a real dependency with a simulated one;
- isolate the business component;
- test only its behaviour.

Dummy and Mock

Context

With dependency injection, real dependencies often need replacing with simplified objects during tests.

Dummy

- Minimal object;
- No useful behaviour;
- Only satisfies a dependency.

Typical use: test a component without side effects (file, network, database).

Mock

- Simulated object;
- Controlled behaviour;
- Verifies interactions.

Typical use: check a component correctly calls a dependency or respects a contract.

Essential difference

Dummy: passive; Mock: active and verifiable.

Example: simple Dummy

Implementation

```
Logger dummyLogger = msg -> {};
Calculator calc =
    new Calculator(dummyLogger);
calc.compute(10); // no disk, no side effect
```

Analysis

- no disk access;
- no side effect;
- fast test.

Example: Mock with verification

Manual mock

```
class TestLogger implements Logger {
    int count = 0;
    public void log(String msg) {
        count++;
    }
}
```

Test

```
TestLogger logger = new TestLogger();
Calculator c =
    new Calculator(logger);
c.compute(10);
assert logger.count > 0;
```

Point

- behaviour verification;
- contract test;
- complete isolation.

Mock frameworks

Observation

Writing mocks by hand quickly becomes tedious.

Solution

Use dedicated frameworks such as Mockito: declare mocks with annotations, verify interactions, no hand-written doubles.

```java
// import static org.mockito.Mockito.*;
@ExtendWith(MockitoExtension.class)
class CalculatorTest {
    @Mock Logger logger;
    @InjectMocks Calculator calc;

    @Test
    void testComputeLogs() {
        calc.compute(10);
        verify(logger, atLeastOnce())
            .log(anyString());
    }
}
```

Final synthesis

To remember

- dependency inversion;
- dependency injection;
- testability;
- automation by framework.

Key message

An injection framework automates a design principle; it does not replace it.

Conclusion

Good design is code that does not know how its dependencies are created.

Section 5

Framework example: Guice

Introduction to Google Guice

Guice

Dependency injection framework for Java
Lightweight, non-intrusive
Based on annotations and Java configuration

Fundamental principle

Guice implements Inversion of Control.

Constructor injection

Class with a dependency

```
public class OrderService {
    private final Database db;
    @Inject
    public OrderService(Database db) {
        this.db = db;
    }
}
```

Why the constructor?

Explicit dependencies
Class valid from creation
Favours immutability

Method injection

Setter injection

```
public class OrderService {
    private Database db;
    @Inject
    public void setDatabase(Database db) {
        this.db = db;
    }
}
```

Usage

Optional dependencies
Late configuration

Guice modules

Module

```
public class AppModule
        extends AbstractModule {
    @Override
    protected void configure() {
        bind(Database.class)
            .to(MySQLDatabase.class);
    }
}
```

Role

Define the concrete implementations
Centralise the assembly

@Provides: controlled creation of dependencies

Module with @Provides

```
public class AppModule
        extends AbstractModule {
    @Provides
    @Singleton
    Database provideDatabase(
            Config config) {
        if (config.isTest()) {
            return new InMemoryDatabase();
        }
        return new MySQLDatabase();
    }
}
```

Role of @Provides

Declares a binding
Allows logic
Parameters injected by Guice

To remember

@Provides expresses assembly logic without breaking IoC.

Creating the injector

Bootstrap

```
Injector injector =
    Guice.createInjector(
        new AppModule());
OrderService service =
    injector.getInstance(
        OrderService.class);
```

Hollywood Principle

The application no longer creates its objects.

Advanced bindings

To an instance

```
bind(Database.class)
    .toInstance(
        new InMemoryDatabase());
```

To a Provider

```
bind(Database.class)
    .toProvider(
        DatabaseProvider.class);
```

Scopes

Singleton

```
@Singleton
public class Database { }
```

Available scopes

By default: new instance
Singleton
Custom scopes

Warning

A Singleton is shared state: keep it stateless, or tests will pollute each other.

Named Bindings

Configuration

```
bind(Database.class)
    .annotatedWith(
        Names.named("prod"))
    .to(MySQLDatabase.class);
```

Injection

```
@Inject
public Service(
    @Named("prod") Database db) {}
```

Optional Bindings

Optional injection

```
@Inject(optional = true)
private Logger logger;
```

Usage

Extensions
Plugins
Conditional configuration

Limits of Guice

Limits

No complete ecosystem
Manual configuration
Not enterprise-oriented

But

Guice is ideal for understanding the basics of IoC.